Tema 2 APD

Ciuca Ionel-Andrei

Todo 1: build_finger_table()

Modalitate de implementare:

Pentru fiecare pozitie i din finger table (i = 0 pana la M – 1)

1. Caculam start[i]
    a. Folosesc o bucla simpla pentru a calcula puterea lui 2^i
2. Gasim finger[i].node cu functia specificata in comentariile cerintei

Todo 2: closest_preceding_finger(key)

Modalitate de implementare:

1. Cautam de la cel mai mare finger, la cel mai mic finger
    a. Dorim sa gasim cel mai mare salt valid, cel mai apropiat de key
2. Verificam intervalul circular deschis
    a. Caz 1 – fara wrap-around
        i. Verificam daca self.id < key
        ii. Verificam daca finger_node > self.id **SI** finger_node < key
    b. Caz 2 – cu warp-around
        i. Verificam self.id > key
        ii. Verificam daca finger_node > self.id **SAU** finger_node < key
3. Returnam primul finger valid gasit, daca exista, daca nu, returnam succesorul

Todo 3: handle_lookup_request(LookupMsg *msg)

Modalitate de implementare:

1. Adaugam nodul curent in traseu prin a marca ca acest nod a procesat mesajul si construim traseul incremental
2. Verificam daca succesorul este responsabil
    a. Daca da, trimitem raspunsul final
    b. Daca nu, forwardam cererea

MAIN

Todo 4: initierea lookup-urilor locale

Modalitate de implementare:

1. Alocam structuri pentru rezultate
    a. results[] – stocheaza rezultatele finale
    b. results_received[] – flags pentru a marca lookup-urile finalizate
    c. lookup_original_keys[] – pastreaza cheile in ordinea din input (folosit ulterior)

2. Salvam cheia originala
    a. Deoarece mesajele MPI pot ajunge in orice ordine, avem neovie de cheile originale pentru a identifica care raspuns corespunde carei cereri

3. Cream si trimitem mesajele
    a. Trimitem la propriul rank pentru uniformitate: toate lookup-urile intra in acelasi flux de procesare
    b. Service loop-ul va primi aceste mesaje si va apela handle_lookup_request()
    c. Primul hop este tratat identic cu orice alt hop

Todo 5: service-loop distribuit

Modalitate de implementare:

1. Variabile de control pentru terminarea distributiei
    a. d_sent = 0 este flag-ul pentru a vedea daca am trimis TAG_DONE
    b. d_received = 0 este pentru a vedea cate mesaje TAG_DONE am primit
    c. all_done = 0 este pentru a vedea daca am terminat lookup-urile
2. Conditia de oprire a loop-ului
    a. Am primit toate raspunsurile si am primit TAG_DONE de la toate celelalte noduri
    b. Avem world_size – 1 deoarece nu trimitem TAG_DONE noua insine
3. Primirea si procesarea mesajelor
    a. Primim de la orice sursa si orice tag
    b. Procesam in funcite de tipul mesajul
        i. TAG_LOOKUP_REQ este cererea de rutare, trimitem mesajul la handle_lookup_request
        ii. TAG_LOOKUP_REP – raspunsul final
            1. Gasim pozitia corecta prin matching dupa cheia originala
            2. Marcam lookup-ul ca finalizat
        iii. TAG_DONE – alt nod a terminat
            1. Un nod anunta ca si-a terminat lookup-urile
            2. Contorizam pentru terminarea globala
        iv. Trimiterea TAG_DONE (o singura data)
            1. Daca all_done (am primit toate raspunsurile) si nu mai am mesaje de trimis
            2. Trimitem la toate celelalte noduri, nu doar la un coordonator
        v. Afisam rezultatele in oridnea corecta
            1. Iteram results[] in ordinea din input
            2. Format Lookup K: n0 -> n1 -> ... ->target
        vi. La final eliberam memoria